

ARCHIVED 2026-08-05 — PROVENANCE RECORD, NOT CURRENT STATE. Kept because the trail is the evidence: what was believed, when, and what broke it. Do not take a number out of this file. `node engine/status.js` is the state. - **Claimed:** that two of the three successor roadmap items describe effects that are not there — in particular that "greedy over-clicks conditional moves" (47.9% / 33.2% / 33.9%) reproduces but is confounded, because the two corpora differ on three levers at once. - **Written:** 2026-08-02, CHANGELOG at 3.31.1. - **Replaced by:** nothing wholesale — this is a dated record. `docs/MODELS.md` and `node engine/status.js` carry what survived. - **Retracted inside:** None registered. The triple it examines was itself withdrawn here and again in `HANDOFF-2026-08-03.md` ; do not re-inherit it as a finding.

Branch `refit-and-semantics-guard` . Suite **56 passed, 1 failed** before this work and after it; the failure is `test-site-data-fresh` , tripped by Will's ingest landing newer game data than the last `engine-data.js` rebuild. Nothing here changed it in either direction.

The previous session's lesson was that all three roadmap items rested on numbers nobody had measured. The same thing happened again to the successors of those items, which is worth saying plainly rather than filing as a coincidence: **two of the three open items describe effects that are not there.**

The claim was "Sucker Punch fails 47.9% under greedy, 33.2% under sampling, 33.9% for humans, at IDENTICAL usage."

It reproduces, and it is confounded. Those two figures come from `games.selfplay` and `games.selfplay-sampling` , which are two separately generated corpora differing on **three** levers at once. Their own stamps say so:

	<code>games.selfplay</code>	<code>games.selfplay-sampling</code>
greedy	true	false
joint	true	false
weights	(default)	policy-weights-clickmatch.json

Attributing the whole gap to greedy required all of the other two to do nothing, which nobody checked.

On a single-lever corpus it disappears. 5,849 games, joint off both arms, the same weights file on both, sides alternating, ~640 Sucker Punches per arm:

	greedy	sampling	human
uses per 1,000 moves	17.2	14.6	14.0
failure rate	42.0%	42.7%	39.8%

Difference **-0.7 points**. Greedy clicks the move more often than a human and does not whiff more when it does.

The noise floor was measured rather than assumed, which is what makes that a result instead of a shrug. Split ONE arm's games at random into two halves — where the true difference is 0.00 by construction — and the 95% band is **[-7.5, +7.0]**. The claimed 14.7-point effect is twice that band. It is not there. `engine/conditional_audit.js --calibrate suckerpunch` re-runs this.

That calibration also explains an oddity found on the way: `games.selfplay` 's two arms are the SAME BUILD and differ by 9.6 points on this statistic, `games.selfplay-sampling` 's by 11.1 in the opposite direction. At those sample sizes the known-zero band is [-10.3, +10.3]. Both were ordinary noise sitting on the boundary — and both were larger than the single-lever effect they were being used to demonstrate.

Greedy still pays, and by far more than this ever was. 87.7% of 269 decisive pairs, DECIDED by `engine/sprt.js` after 38.

The claim was "sampling still Protects 1.44x the human rate, so roughly half the excess is in the weights, not in greedy."

The rate is real and the diagnosis is wrong. Measured on the single-lever corpus: greedy 215.2 per 1,000 moves, sampling 159.9, human 128.5 — so sampling is **1.24x**, not 1.44x, and 36% of the excess survives sampling rather than "roughly half".

The weights are not the cause, and this is a sharp test rather than an argument. A conditional logit at its own optimum reproduces the observed count of any subset its features can express — the log-likelihood gradient is exactly (observed - predicted) summed over rows. So score the shipped vector against the human decisions it was fitted on, holding the choice sets fixed (`engine/protect_calibration.js`):

on 11,517 human decisions that offered a Protect	
human pressed one	18.86%
model expects one	19.22%
ratio	1.019

The model reproduces the human Protect rate to within 2%. The in-play excess is therefore the argmax on one side and **distribution shift** on the other — MAG reaching Protect-favourable boards more often than the fit corpus contains. Reweighting cannot fix a distribution.

What the 1.019 is hiding is the actual finding. It is a near-perfect average of large, offsetting per-species errors:

species	human	model	gap
Gengar	40.6%	21.6%	-19.0
Blaziken	33.6%	15.7%	-18.0
Mamoswine	26.8%	13.6%	-13.2
Frolass	8.5%	20.6%	+12.0
Farigiraf	7.6%	19.4%	+11.8
Archaludon	28.6%	18.8%	-9.8

There is no species-conditional Protect term in the vector, so the model **Protects the wrong Pokemon at the right overall rate**. On the fit corpus the errors cancel by construction. In self-play, with different teams and different states, they stop cancelling and the aggregate blows out. That is a missing FEATURE, not a weight to tune, and it is the one thing here that is actually actionable.

And the excess Protects are worse Protects, which nothing had measured. Share that block anything:

	greedy	sampling	human
Protect	59.0%	54.9%	70.6%
Wide Guard	18.1%	21.4%	44.9%
Quick Guard	4.5%	4.6%	27.0%

Quick Guard at 4.5% is close to pure waste.

The claim was "MAG under-switches, and it costs a third of the self-play corpus to `partial_bring` — dropped games average 2.86 switch events against 4.25 kept."

The evidence was circular. A third Pokemon appears because something switched it in, so "bring count" and "switch count" are very nearly the same quantity. "Dropped games have fewer switches" restates "dropped games have fewer mons brought", which is the definition of the filter.

And `partial_bring` is not a switching detector at all. In **1,008 of 1,008** dropped self-play games the short side is the **WINNER**; the loser is short in zero of them. The loser's replacements are forced by faints, so the loser always reaches four. It is a **decisiveness filter** — it discards the games MAG won without losing two Pokemon (mean 8.55 turns against 12.28 for the games it keeps). The human corpus has the same shape: 7,349 winner-short against 267 loser-short.

That is worth knowing on its own: `require_full_bring` systematically removes fast, dominant wins from the corpus a value function learns positions from.

The real finding is that the lever was never on. Voluntary switches — any `|switch|` before the turn's first `|move|`, since a voluntary switch resolves at +6 priority:

corpus	games	pre-move switches	minus 4/game send-outs
games.selfplay	2,901	11,604	0
games.selfplay-sampling	1,174	4,696	0
games.h2h-greedy-vs-sample	600	2,400	0
games.ladder (human)	27,953	151,293	39,481

Exactly `4 x games` in all three, to the individual count, because every self-play corpus on disk was generated `switching: false` and the candidate list contained no switch to pick. This is the `ABRA_STRICT_SEMANTICS` shape again: a capability that was implemented, documented, and never switched on.

Turned on, it wins — narrowly, and it took real evidence to say so.

switching ON vs OFF, both greedy, paired 54.1% of 1,032 decisive pairs DECIDED after 899, accept H1

Compare greedy, which decided after 38. A run sized by the usual "200-400 decisive pairs" would have stopped this one undecided at 51.2% and concluded the wrong thing; the SPRT was read as it went across four batches and only crossed at 20,630 games.

The behavioural gap is far from closed. With the lever on MAG makes **28.6** voluntary switches per 1,000 turns against a human **322.3** — 11x fewer. The lever is not the constraint any more; how the policy scores a switch is.

-
- `engine/conditional_audit.js` — the missing tool behind the 47.9%. Derives the 39 conditional-on-the-opponent moves from the dex's own handlers (`queue.willMove` , `queue.willAct` , `activeMoveActions` , `newlySwitched`); no move is named in the file. Reports per ARM, through each record's own `swapped` stamp, and carries `--calibrate` so its interval is checked against a known zero rather than trusted.
 - `engine/protect_calibration.js` — scores the shipped vector against the corpus it was fitted on.
 - `engine/realism_report.js` — "switches per game" split into VOLUNTARY and forced, per 1,000 turns. That one line read 8.23 against 10.76 and was not even flagged large; split, it reads **0.00 against 322.25**, the biggest gap in the table. It also now prints the corpus's lever stamp and refuses to let a lever-off metric read as a policy measurement.
 - `engine/feature_coverage.js` — "no switches observed" and "no observations at all" were the same message. A corpus without `--thoughts` made it announce "THE PLAYER CANNOT SWITCH" from an empty sample. Separated.
 - `engine/fit_policy.js` — decisions now carry `mvs` , the candidate identities, so a question ABOUT the fit does not need a second replay loop (R2).

- `tests/test-docs-current.js` — a new check that derives a claim's licence from an artifact instead of from a hand-maintained list: no document may assert a non-transitive metagame while `data/slowking-playstyle-eval.json` records its own best cycle `supported: false`. It found three (`MODELS.md` x2, `ROADMAP.md`), now qualified. It distinguishes a document QUOTING the claim in order to retract it — two of the five initial hits were the reviews that did the right thing.
- `engine/magnemite.js` — the greedy comment quoted `isStatus +0.50`, `protectThreatened +0.49`, `accuracy -1.51`. Post-refit those are `+0.26`, `+0.32`, `-0.795`, and the behavioural inference drawn from them is disproved by the calibration above.

The two watched numbers did not move: **6 declared misses, 239 silent catches, 0 new** (four of mine were caught by `test-no-silent-failure.js` and fixed rather than baselined).

Last session: six documented claims failed measurement. This session: three of the four items I was handed did, and the fourth (the playstyle cycle) had already been caught by the previous session's own test rewrite.

The shape is always the same and it is not carelessness. Every one of these numbers was **measured** by somebody. What was missing was the thing that makes a measurement a comparison:

- **the 47.9%** — real numbers from two corpora that differed in three ways
- **the 1.44x** — a real rate against a human baseline computed differently
- **the 2.86 vs 4.25** — two real averages of what is nearly the same quantity
- **the cycle** — a real 3-cycle, and the strongest of 336 candidate triples

The countermeasure that worked here was not more care. It was **measuring the noise floor** — split one arm in half, where the answer is known to be zero, and see how big a difference the instrument invents. Do that first and three of these four die in a minute, with no argument required.

1. **The species-conditional Protect gap** (§2). The only genuinely actionable finding here: the model has no term for WHO is holding the Protect, and the per-species errors are `+12` to `-19` points.
2. **The page swap** — unchanged from this morning, still a cost decision, not engineering.
3. **How the policy scores a switch** (§3). The lever pays 54.1% and MAG still switches 11x less than a human, so the headroom is in the switch features rather than in the flag.
4. `require_full_bring` **discards decisive wins** (§3). Stated, not fixed — it is a real selection bias on the corpus a value function trains on, and changing it is Will's call.
5. **Extend the lookup contract to the remaining accessors** — unchanged.

7. THE THING ALL OF IT WAS, NAMED BY WILL

"ITS PART OF THE MULTI TURN TAILWIND TYPE ANALYSIS" — and it is.

Three Protect anomalies were measured separately and treated as separate. They are one thing: **the feature vector prices ONE TURN**. Value that accrues over several has nowhere to go.

Measured over 58 species with `>= 30` Protect-offering decisions, splitting status moves by whether the dex gives them a duration (`sideCondition` , `pseudoWeather` , `weather` , `terrain` , `slotCondition` , `condition.duration` — nothing named):

<code>corr(Protect residual , MULTI-TURN moves on the sheet)</code>	-0.343
<code>corr(Protect residual , INSTANT status moves)</code>	+0.066

multi-turn moves on the sheet	mean residual
0 (n=36)	+0.8 pts
1 (n=15)	-1.3 pts
2 (n=7)	-4.9 pts

The entire effect sits in the duration moves. Follow Me, Fake Out and Taunt carry none of it. And isolating the mechanism IMPROVED the correlation — -0.265 across all support moves, **-0.343** across multi-turn only — which is what a real mechanism does and a proxy does not.

Both directions are the same missing quantity:

	the multi-turn value	the model
Blaziken Protects	Speed Boost banks +1 Speed for later turns	under-predicts, -18.0
Gengar Protects	burns Perish turns while Shadow Tag traps	under-predicts, -19.0
Pelipper / Politoed / Farigiraf do NOT	Tailwind, Trick Room, screens are the better investment	over-predicts, +10 to +12

They cancel, which is why the global ratio reads 1.021 and looks calibrated. An aggregate that looks right because two real errors point opposite ways is the shape of this entire session.

Corroboration that was sitting in plain sight. `terrainSetupHelpsPartner` **+1.606** and `weatherSetupHelpsPartner` **+1.311** are the two largest weights in the whole 18-term joint block. The pair fit found multi-turn setup value the marginal vector cannot represent, and routed it through the only terms available to it. That was filed as "three pair weights are barely observed" and read as a curiosity.

`engine/mechanics_coverage.js` already describes the model as "*conditional, one-step-ahead mechanics, and a static feature vector describing the board as*" it is now. That sentence was written as a description. It is the binding constraint.

What is ruled out, so it is not re-inherited. The Protect gap is NOT: priorLogP underweighted (corr 0.032 with the residual), a population mismatch in `move-priors.json` (it matches the fit corpus to three decimals), Choice-lock (5 of 11,517 decisions), or the mega ability gap — that was fixed universally this session and the residuals did not move a decimal place. Each was measured and each was wrong, including two I was confident about.

Do not chase the Protect rate. It is a symptom of a one-step scorer, and the fix is a feature that can hold accrued value, not a Protect term.